

RarePlanes Detection Using YOLOv5 on the Versal Adaptive SoC

Jacob Brown, Colton Yates, Jeffrey Goeders, and Michael Wirthlin
Brigham Young University
450 Engineering Building
Provo, UT 84602
{jacobbb00, coltony, jgoeders, wirthlin}@byu.edu

Abstract—Object detection is an important operation for many satellites performing remote sensing operations. Machine learning models are increasingly used for real-time object detection. It is crucial that such machine learning models perform inference on satellite images accurately and reliably, while meeting power, size, and other constraints. We train several YOLOv5 object detection models on the RarePlanes dataset, a collection of satellite aerial images of airplanes, and deploy the trained models to the Deep Learning Processing Unit on the Versal AI Core XCV1902 device. The performance and accuracy of the models running on the Versal device are measured and compared to implementations on a CPU and GPU. Radiation-induced single-event upsets are then simulated by injecting faults into the programmable logic of the Versal device, and the effects of each fault on the YOLOv5 model are discovered.

TABLE OF CONTENTS

1. INTRODUCTION.....	1
2. RELATED WORK	1
3. YOLOv5 OBJECT DETECTION WITH RAREPLANES2	
4. VERSAL ADAPTIVE SOC	3
5. YOLOv5 ON VERSAL	4
6. FAULT INJECTION	5
7. CONCLUSION	7
ACKNOWLEDGMENTS	8

1. INTRODUCTION

Object detection is an important operation for satellites to be able to perform, and in outer-space missions it is crucial that machine learning models perform inference on satellite images accurately and reliably. Where size, power, and other constraints exist, meeting this goal for accurate and reliable object detection is challenging. Additionally, soft errors caused by radiation can disrupt object detection algorithms, resulting in incorrect object classifications, false object detection, or system-level errors or hangs. This work investigates the performance of object detection models on an embedded device in the presence of soft errors.

We used the well-known and high-performing YOLOv5 object detection model, and trained various model sizes of YOLOv5 on a collection of satellite images of airplanes known as the RarePlanes dataset. The target device is the AMD XCV1902 Versal Adaptive SoC device operating on

the VCK190 evaluation board. The Versal part was chosen as the deployment target because it is capable of high performance computing at low power due to its AI engines and programmable logic (PL) resources. This particular device has a space grade variant, the XQRVC1902, which makes it an attractive platform for space applications. While our experiments do not use the space grade version, we anticipate the results to be the same as the space grade variant uses the same architecture as the device used in these experiments.

In this work, our trained models were quantized and compiled using AMD’s Vitis AI framework and targeted to the Deep Learning Processing Unit (DPU), an intellectual property (IP) module provided by AMD. Performance of the YOLOv5 models on the DPU varies based on model size and DPU configuration, with a range of frame rates from 20 fps to 300 fps. All model and DPU configurations result in a higher framerate than a standard desktop CPU running the same models. Accuracy of the floating point YOLOv5 models ranges from moderate to high, with F1 scores ranging from 0.845 to 0.911. Accuracy differences between the original 32-bit floating-point models and the 8-bit integer quantized models are minimal, with the greatest change in F1 score being about 0.02.

The focus of this work is to investigate the impact of soft errors on the YOLOv5 models running on the DPU. We developed a test framework to simulate single event upsets (SEUs) in the programmable logic of the Versal device while the YOLOv5 models are running. The impact of these upsets on the object detection results are measured and analyzed. Benchmarking tests were executed as faults were injected, and results were compared to the golden results recorded before any faults were injected. Over several weeks, more than 400,000 faults were injected, of which 7.48% had noticeable effects. Of the faults with noticeable effects, 30% had tolerable effects (slightly altered but still correct predictions) and 70% had non-tolerable effects (incorrect predictions, DPU timeouts, and CPU hangs).

2. RELATED WORK

In recent years, FPGA devices have commonly been used to accelerate machine learning models, particularly convolutional neural networks (CNNs). FPGAs typically provide much higher performance than CPUs and lower power consumption than GPUs. Several projects have been developed to accelerate CNNs on FPGAs. In [1], the authors present the FINN tool which demonstrates the feasibility and effectiveness of implementing reduced-precision CNNs on FPGAs. More recently, Jia et al. [2] and Perryman et al. [3] have demonstrated deploying image classification CNNs to the

J. Brown, C. Yates, J. Goeders, and M. Wirthlin are also with the NSF Center for Space, High-performance, and Resilient Computing (SHREC). This work was funded by the National Science Foundation under Grant Number 1738550.

AMD Versal device. Other recent works have demonstrated implementing the YOLOv5 model on FPGAs, including Saidani et al. [4], who deployed the model on an AMD Zynq-7000 SoC with 120 fps, and Lyu et al. [5], who implemented the model on an AMD Zynq UltraScale+ MPSoC with 6 fps and only 10 W of power.

With CNNs being deployed to FPGAs, interest in the effect of radiation-induced single-event upsets (SEUs) on FPGAs running CNNs has increased. Recent works that have investigated the effects of SEUs on CNNs deployed to FPGAs include [6] where 0.71% of faults caused classification errors, [7] where about 5% of injected faults affected model operation but only 0.2% caused misclassifications, and [8] where about 1.8% of faults injected into the FPGA caused misclassifications. Libano et al. [9] found that quantizing their CNN model increased the number of error-causing faults by 12%. While investigating ways to mitigate soft errors in FPGAs running CNNs, Li et al. [10] found that SEUs in processing elements such as adders and multipliers affect CNNs the most. In an effort to mitigate SEUs at minimal cost, Libano et al. [11] selectively replicated their CNN model accelerator and was able to mask 40% of errors with an overhead of only 8%.

3. YOLOv5 OBJECT DETECTION WITH RAREPLANES

YOLOv5, or You Only Look Once version 5, is a well-known machine learning model capable of image classification, object detection, and instance segmentation. Due to its architecture and ability to propagate an image through the model only a single time, YOLOv5 boasts lower inference times while achieving similar accuracy when compared to other machine learning models. The YOLOv5 framework, provided by Ultralytics [12], is open-source and built on PyTorch [13]. With its extensive tutorials and documentation, YOLOv5 allows beginners to easily train a model on custom datasets and deploy the model in their environment. YOLOv5 also provides a variety of model sizes; smaller models have a faster inference time while larger models have a higher accuracy. This allows the user to select the model that provides the desired balance between speed and accuracy.

The RarePlanes dataset [14] is a collection of real and synthetic satellite aerial images of airplanes. Each image contains one or more airplanes of various types, with a companion label file indicating size, type, and use of each aircraft. The RarePlanes dataset was chosen because it is representative of images that may be taken from satellites. Only the real images were used in this work while the synthetic images were left out, resulting in train, test, and validation datasets of size 5815, 2590, and 120 images, respectively. The original images are size 512x512, but were resized to 640x640 during training as per the YOLOv5 default.

Several sizes of the YOLOv5 model were trained on the RarePlanes dataset using an NVIDIA GeForce RTX 3060 GPU. Additionally, two legacy YOLOv3 model configurations were also trained, and are included in the model set, even when the term “YOLOv5 models” is used. All the default training hyperparameters were used except for number of epochs, which was set to 600 to ensure the models were trained as much as possible. The largest model took about 14 hours to train, and the smallest model took about 3 hours to train. Although number of epochs was set to 600, every training session stopped before reaching that point

due to YOLOv5’s EarlyStopping feature. Also, YOLOv5 uses the Sigmoid Linear Unit (SiLU) activation function, but for reasons explained in a later section, all instances of the SiLU activation function were replaced with LeakyReLU, another type of activation function, with a negative slope of 0.1015625. Figure 1 demonstrates the trained YOLOv5m model’s detections for one of the RarePlanes images from the test dataset. In this example, three “Large Civil Transport” planes are correctly detected and labeled.



Figure 1: YOLOv5m inference on an image

The performance of each YOLOv5 model trained on the RarePlanes dataset was evaluated on the test dataset after training. We focus on two metrics: F1 score to measure accuracy (described in more detail below) and frames per second (fps) to measure throughput. Table 1 shows the throughput performance of each of the YOLO models on both a CPU and a GPU. The CPU is a 12th generation Intel i9 with 64 GB of memory running Ubuntu 20.04, and the GPU is an NVIDIA GeForce RTX 4090. Note that the confidence and Non-Max Suppression (NMS) thresholds used during inference were 0.70 and 0.45, respectively. NMS is a postprocessing technique used to eliminate duplicate detections by removing bounding boxes whose percentage of overlap with other bounding boxes is above a certain threshold. The confidence threshold is used to remove any predictions with confidences lower than that threshold. As expected, the smallest model, YOLOv5n, has the highest fps and the largest model, YOLOv5x, has the lowest fps. Further, the GPU demonstrates a higher performance than the CPU with a geometric mean of 24 times that of the CPU.

The F1 score (Equation 1) is a value between 0 and 1 that is used to measure model accuracy. This score is the harmonic mean between precision and recall, with 1 being the highest accuracy possible. Precision measures how many of the total predictions are correct and recall measures how many objects in the image are correctly identified. This measure includes the effect of both false positives and false negatives. Because a model’s accuracy varies due to the confidence threshold used, many thresholds were tested (a lower threshold will result in more detections and thus more false positives, whereas a higher threshold will result in fewer detections and thus

Model	CPU (fps)	GPU (fps)
YOLOv5n	61.89	356.91
YOLOv5s	23.23	342.97
YOLOv5m	9.04	255.84
YOLOv5l	4.47	196.07
YOLOv5x	2.41	116.97
YOLOv3-tiny	27.83	476.16
YOLOv3-spp	3.06	168.00

Table 1: CPU and GPU inference speed for different YOLO models in fps.

Model	Prec.	Recall	F1	Conf. Thresh
YOLOv5n	0.885	0.862	0.873	0.45
YOLOv5s	0.890	0.882	0.886	0.45
YOLOv5m	0.909	0.893	0.901	0.55
YOLOv5l	0.917	0.895	0.906	0.60
YOLOv5x	0.924	0.898	0.911	0.60
YOLOv3-tiny	0.841	0.850	0.845	0.35
YOLOv3-spp	0.904	0.872	0.888	0.55

Table 2: Accuracy metrics for various YOLO models (Precision, Recall, F1 Score, and Confidence Threshold).

more false negatives). Table 2 gives the highest accuracy of each model and what confidence threshold was used to obtain the given accuracy. Like the throughput test detailed earlier, the NMS threshold used was 0.45.

$$F1 = 2 * \frac{Precision * Recall}{Precision + Recall} \quad (1)$$

As expected, the larger models have a higher accuracy than the smaller models. Generally, an F1 score greater than 0.9 is considered excellent, an F1 score between 0.8 and 0.9 is considered good, and an F1 score between 0.7 and 0.8 is considered okay. Following these general guidelines, our YOLOv5 models have good and excellent F1 scores.

4. VERSAL ADAPTIVE SOC

As new machine learning models such as YOLOv5 are developed, new technologies designed to support machine learning applications are being released. One device is AMD’s Versal Adaptive SoC, or Versal ASoC [15], which combines ARM processors, programmable logic, and AI Engines all connected by a programmable Network on Chip (NoC). A key goal of the Versal device is to support the acceleration of many applications including autonomous vehicles to networking to medical equipment, while meeting strict power and size constraints.

The AI engine section of the device is an array made up of AI engine tiles. Each tile is composed of an AI engine core, local memory, AXI memory interfaces to neighboring tiles, and an interconnect to interface with non-neighbor tiles using AXI stream. The AI engine core contains both a scalar and vector unit, the vector being a 512 bit very-long-instruction-word (VLIW) single-instruction-multiple-data (SIMD) processor.

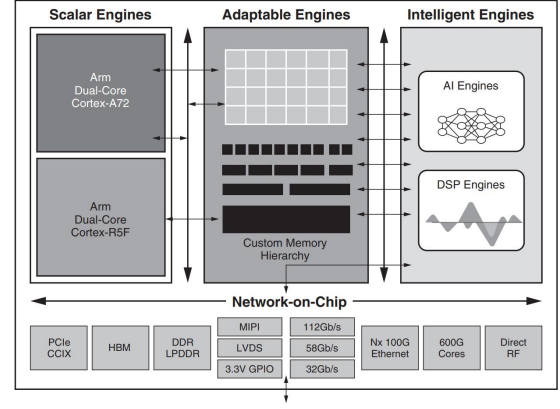


Figure 2: Versal ASoC Functional Diagram [16]

Each tile has 32KB of data memory and 16KB of program memory, and can directly access memory of immediate neighbors, resulting in 128KB of data memory available. The AI engine array provides three levels of parallelism: 1) SIMD, 2) VLIW, and 3) all AI engines can execute in parallel.

The VCK190 development board from AMD contains the Versal AI Core XCV1902 device (XCV1902-2MSEVSVA2197) and was used for the experiments completed in this paper. This device has almost 2000K logic cells and 400 AI Engines [17]. The board is equipped with Ethernet and USB interfaces to aid in evaluation and development. Additionally, there is an external system controller Zynq UltraScale+ MPSoC onboard that provides an interface to monitor power, clocks, and other resources of the Versal device.

The Deep Learning Processing Unit (DPU) is intellectual property provided by AMD that provides a computation engine designed to run convolutional neural networks (CNNs). The specific DPU IP for the XCV1902 part is the DPUCVDX8G [18] and consists of programmable logic and AI Engines, as shown in Figure 3. The DPU supports common CNN operations such as convolution, pooling, and fully connected layers. Although the AI Engines are capable of many operations, the DPU employs them only for convolution, and the other operations are performed in the programmable logic. The DPU IP is configurable, allowing the user to select a batch size anywhere from 1 to 6, and 32 or 64 AI Engines for each batch handler. Batch size is how many images are processed in the CNN at once, so a higher batch size gives a higher throughput. The highest parallelism is achieved with a batch size of 5 and 64 AI Engines, totalling 320 AI Engines and a theoretical peak performance of 102.4 TOPS when the AI Engines run at 1.25 GHz and the DPU clock frequency is 333 MHz. When configuring the DPU IP, the user also has an option to have more than one DPU compute unit which allows running multiple models simultaneously. Using more than one DPU limits the batch size to 1.

To support developers building machine learning applications on the DPU IP, AMD provides Vitis AI, an open-source project found on GitHub. Vitis AI [19] is a toolset that takes machine learning models from popular frameworks such as Pytorch and Tensorflow [20] and prepares them for deployment on the DPU, allowing developers to build and train their own models using their preferred framework and

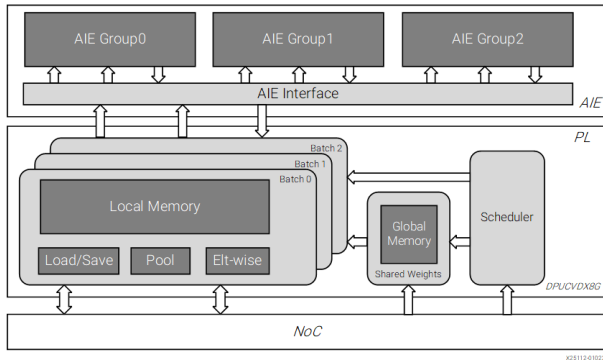


Figure 3: DPUCVDX8G Block Diagram

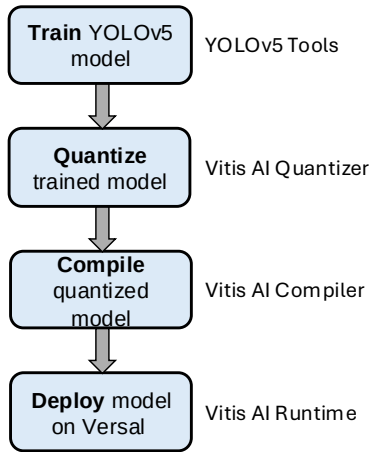


Figure 4: Flow to Deploy YOLOv5 on Versal

still be able to run them on AMD’s DPU. The process of quantizing a 32-bit floating-point model to an 8-bit integer format is handled by the Vitis AI Quantizer, and then the Vitis AI Compiler converts the model into DPU instructions. To load the compiled model onto the DPU and run inference, Vitis AI Runtime is used.

5. YOLOv5 ON VERSAL

To deploy our YOLOv5 models that were trained on the RarePlanes dataset to the Versal device on the VCK190 board, we followed the flow shown in Figure 4. The models are trained on the RarePlanes dataset, quantized, compiled, and run on the Versal device.

The Petalinux image booted on the board was built using the reference design provided in the Vitis AI repository checked out to tag 3.0, which builds all needed hardware and software for running models on the DPU. Building the design requires AMD Vivado, Vitis, and Petalinux 2022.2, and the output board image can be written to an SD card and booted on the VCK190 board. The DPUCVDX8G IP version is 1.0, the Versal CIPS version is 3.3, the NOC IP version is 1.0, and the AI Engine IP version is 2.0.

In order to deploy the YOLOv5 model to the Versal DPU, the model has to be modified. In particular, all instances of SiLU in YOLOv5 were replaced with LeakyReLU. The

reason for this is that the DPUCVDX8G does not support the SiLU activation function, and thus must be changed to LeakyReLU with a negative slope of 0.1015625. The models trained in a previous section have been modified to support this change. To use the Vitis AI tools to quantize and compile a model, the default Vitis AI PyTorch docker image was used. While most of the YOLOv5 model can run on the DPU, some operations in the final steps are not supported by the DPU; so when quantizing, the code in the *Detect* class in *yolov5/models/yolo.py* must be modified to only execute the convolution layers. The remaining operations must be performed on the CPU after the DPU runs.

Neural network models contain a large number of weights and biases stored in floating-point numbers. To reduce memory and computational costs, quantization is used to reduce the size of a model while minimally affecting accuracy. The Vitis AI Quantizer supports both Post Training Quantization (PTQ) and Quantization Aware Training (QAT), and we used PTQ because the accuracy loss due to quantization was minimal, with the greatest drop in F1 accuracy being 0.021 (see Table 3). The 32-bit floating point YOLOv5 model is quantized to an 8-bit integer model and exported in a Xilinx Intermediate Representation (XIR) based format. When quantizing the YOLOv5 models, all default quantization settings were used. The Vitis AI Compiler then takes the XIR model output by the quantizer and generates DPU instructions, also in an XIR-based format. An important input to the compiler is a file specifying the architecture of the DPU. Such a file is generated when building the board image as described above. Models compiled for one DPU target will not run on another.

To run a compiled model through the DPU on the Versal device, Vitis AI Runtime (VART) is used. VART is an API for loading and running models on the DPU in just a few lines of code. In general, the overall flow of a script is as follows: 1) load the compiled model into a VART DPU Runner object, 2) load and preprocess sample images, 3) fill the DPU Runner’s input tensors with the preprocessed images, 4) wait for the DPU Runner to finish and obtain DPU Runner output tensors, and 5) perform any postprocessing necessary. In the case of our YOLOv5 models, image preprocessing includes resizing images to 640x640 and normalizing to values between 0 and 1, and post processing includes performing the final operations in the *Detect* head, as explained previously.

VART has both C++ and Python interfaces, and because YOLOv5 is originally built on PyTorch, our first attempt at running a YOLOv5 model on the board was with a Python script. PyTorch was installed on the board and used to perform image preprocessing and the final model postprocessing, but this proved to be quite slow. Our second attempt involved using VART’s C++ interface and PyTorch’s C++ library, LibTorch and LibTorchVision, and this resulted in improved throughput. The final attempt was to use a YOLOv5 script provided by AMD and found in the Vitis AI library, which is a C++ program that does all pre- and post-processing using OpenCV and custom functions without using LibTorch or LibTorchVision. Using this script resulted in the best throughput of the model on the DPU. The changing factor between all three scripts is the image preprocessing and model postprocessing performed on the CPU while the DPU model processing remains the same.

The performance and accuracy of each YOLOv5 model was evaluated on various DPU configurations on the VCK190 board. As with the CPU test described above, we focus

Model	Version	Prec.	Recall	F1	Conf. Thresh	F1 Reduction
YOLOv5n	Float	0.885	0.862	0.873	0.45	
YOLOv5n	Quant	0.848	0.855	0.852	0.40	0.021
YOLOv5s	Float	0.890	0.882	0.886	0.45	
YOLOv5s	Quant	0.887	0.876	0.881	0.40	0.005
YOLOv5m	Float	0.909	0.893	0.901	0.55	
YOLOv5m	Quant	0.909	0.892	0.901	0.50	0.000
YOLOv5l	Float	0.917	0.895	0.906	0.60	
YOLOv5l	Quant	0.913	0.893	0.903	0.50	0.003
YOLOv5x	Float	0.924	0.898	0.911	0.60	
YOLOv5x	Quant	0.920	0.894	0.907	0.50	0.004
YOLOv3-tiny	Float	0.841	0.850	0.845	0.35	
YOLOv3-tiny	Quant	0.850	0.835	0.843	0.40	0.002
YOLOv3-spp	Float	0.904	0.872	0.888	0.55	
YOLOv3-spp	Quant	0.895	0.861	0.878	0.45	0.010

Table 3: Comparison of Float and Quantized YOLOv5 Models

on throughput (fps) and accuracy (F1) and set the NMS threshold to 0.45 and confidence threshold to 0.70. Table 4 summarizes the results of the models on the DPU, with the CPU and GPU results included again for comparison. All DPU configurations have one compute unit, and the only changing parameters are number of AI Engines and batch size.

As expected, the more batch handlers in the DPU, the higher the throughput. Although configurations with 64 AI Engines theoretically provide more parallelism than configurations with 32 AI Engines, the latter sometimes outperforms the former with YOLOv5n and YOLOv5s, indicating that the overhead of more parallelism in the DPU does not always provide a higher throughput. Additionally, the larger the model is, the more it benefits from the DPU having additional AI Engines. It is also interesting to note that the increase in throughput between configurations with 32 and 64 AI Engines is greatest when the batch size is 1. A “smaller” DPU may perform just as well as a “larger” DPU while using less programmable logic and AI Engine resources, depending on the application.

All DPU configurations outperform the CPU. The greatest increase from CPU to DPU was in YOLOv5x with an improvement of 29.8x. The NVIDIA GPU outperforms the DPU in every instance posed, but the power consumption of the GPU is much higher than that of the Versal device. During inference using the YOLOv5m model, the GPU had an average power of 110 W and a peak power of 113 W (obtained using the `nvidia-smi` tool), while the Versal device had an average power of 18 W and a peak power of 20 W (obtained using the VCK190 external system controller) for the C32B5 DPU configuration, resulting in the fps per Watt ratio for the DPU being three times that of the GPU.

6. FAULT INJECTION

To study the performance of the VC1902 running YOLOv5 object detection models in a high-radiation environment, we simulated the effects of radiation-induced single-event upsets (SEUs) by injecting faults into the programmable logic CRAM of the device. The DPU configuration used during our fault injection tests was C32B5.

To inject faults into the programmable logic, we use the AMD Soft Error Mitigation (XilSEM) library, which is part of the Platform Loader and Manager (PLM) firmware. To invoke this function in the PLM, we generate a Programmable Device Image (PDI) with the necessary function call and use the AMD `fpgautil` tool to send the PDI to the PLM.

The same benchmarking test for the YOLOv5m model used in the previous section is used here to exercise the DPU after each fault is injected, but inference is only performed on 25 images rather than the entire test dataset. The 25 images used were randomly selected from the test dataset. It is important to note that the model output for an image is deterministic, so we can expect the output to never change unless a fault causes a disruption.

The flow for our fault injection testing is shown in Figure 5. First, an initial benchmark test is run to determine the expected output of the DPU running inference on the selected 25 images, and this initial test output is referred to as the “golden” output. Then, XilSEM is disabled so faults will not be corrected, a random fault injected, and the same benchmark test run. The output of the benchmark test is compared with the golden output, and any differences are classified as errors. Occasionally, we have observed a fault that only sometimes causes an error i.e. the first run output matches the golden copy but the second run does not, so we run the benchmark test twice for each fault to help us not miss these types of errors. After the fault is tested for effects, XilSEM is enabled and the fault corrected. If no error was observed during the benchmark test, another random fault is injected and the process repeated. However, if an error was observed, the benchmarking test is rerun twice after XilSEM corrects the fault to ensure the effect of the fault is not persistent, as was sometimes encountered.

Sometimes a fault causes an error that requires the board to reboot. One such error is known as a DPU timeout, and causes the DPU to not be able to run a model. Another is when a fault’s effects persist after the fault is corrected, as described before. With these errors, the board can reboot itself. With the last type of error that requires a reboot, a CPU hang, an external host is required to watch for when this happens and reboot the board using the external system controller. A host is used to SSH into the board and perform

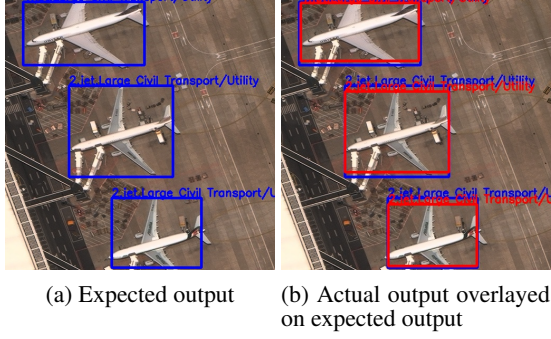


Figure 7: Example of fault with tolerable effects

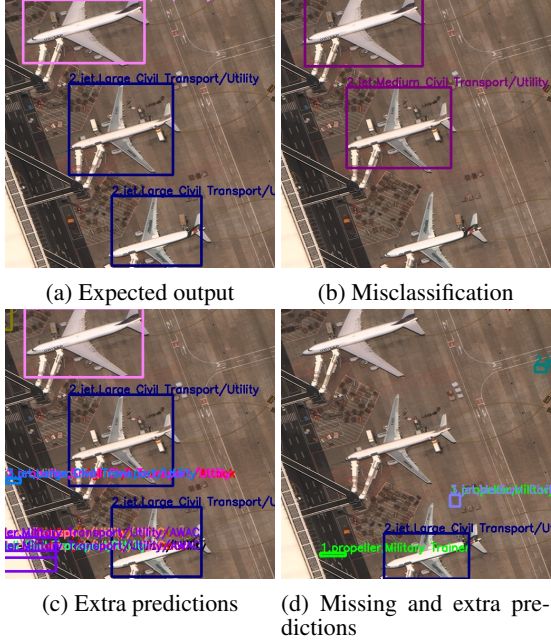


Figure 8: Examples of faults causing incorrect predictions

handlers. By counting how many images each fault affected, it becomes clear that most faults affected exactly 5 of the 25 images (see Figure 9), and upon further examination, we find that 99% percent of these faults affect one image group, meaning one batch handler is affected while the others continue to operate correctly. Similar findings were found with faults that affected 10, 15, and 20 images, indicating that the faults affect 2, 3, and 4 batch handlers, respectively, though less often. It is not possible to distinguish whether faults that affected all 25 images affected all 5 batch handlers or some other part of the DPU.

Looking at the next tallest bar in Figure 9, faults that affected 4 images, a similar situation is found. For faults that affected 4 images, 99% of the time, the 4 images are from the same image group. In fact, the same is true in the situations where 2 or 3 images were affected. These faults are affecting the operation of one batch handler only part of the time.

Another way to look at the faults in the “Incorrect Results” category is to see how they affected the number of predictions for each image. For each fault, the prediction results of each of the 25 images was checked to see if the number of predictions increased, decreased, or was unchanged. 0.13%

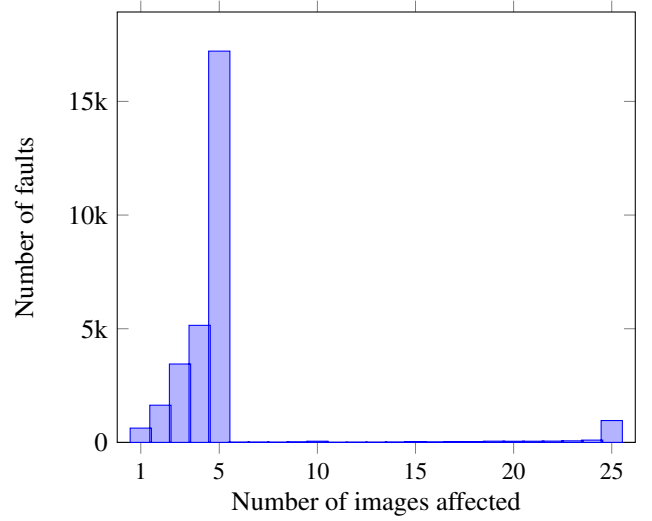


Figure 9: Bar chart showing how many faults affected X number of images

of faults caused an increase in number of objects predicted across all 25 images, while 0.74% caused a decrease. 14.69% of faults caused an increase in predictions for some of the images and no change in the others, while 38.83% caused a decrease in predictions in some images and no change in the others. 9.89% of faults caused both an increase in some images and decrease in others, and 35.74% caused no change in the number of predictions in any of the images. These results are contained in Table 7. We see that most faults caused a decrease in the number of predictions in some of the images or no change for any of the images.

Category	Percentage
All increased	0.13%
All decreased	0.74%
Only increased	14.69%
Only decreased	38.83%
Split	9.89%
Nothing	35.74%

Table 7: Percentage of faults that caused changes in the number of predictions for the 25 images

Although the DPU is powerful IP that can run YOLOv5 models at a high throughput (as shown in the previous section), our fault injection experiments show that a substantial percentage of bit faults can affect the operation of the DPU and, by extension, the YOLOv5 model.

7. CONCLUSION

In this work, we investigated the performance of a well-known object detection model, YOLOv5, on the Versal Adaptive SoC, a device designed to deliver high performance computing at lower power than traditional GPUs. We trained several YOLOv5 models on the RarePlanes dataset, a collection of satellite aerial images of airplanes, and found they perform well by measuring their F1 scores. Next, we used the Vitis AI tools to quantize the models into an 8-bit integer format and deploy them to the DPU running on the XCVC1902 device.

The performance and accuracy of each model running on the DPU was measured for several DPU configurations. In all instances, the DPU provided a higher model throughput than a traditional CPU. Finally, we simulated the effects of single-event upsets caused by radiation on the operation of the DPU running the YOLOv5 models by injecting faults into the programmable logic of the Versal device. We found that 7% of faults cause a noticeable change in DPU operation, with 4.46% of faults causing nontolerable effects.

We are planning to test the behavior of the design in a radiation environment to validate the fault injection results and will publish the results of this experiment in a future publication. We may also perform similar experiments with the VEK280 evaluation kit by AMD, a board that features XCVE2802 device which is part of the Versal AI Edge series and is more suited for low-power applications. Other future work may include custom hardware acceleration kernels to perform YOLOv5 postprocessing in the programmable logic, thus improving model throughput.

ACKNOWLEDGMENTS

This work was supported by the I/UCRC Program of the National Science Foundation under Grant No. 1738550 through the NSF Center for Space, High-performance, and Resilient Computing (SHREC). We also acknowledge the donation of the VCK190 board for this work by the AMD University Program.

REFERENCES

- [1] Y. Umuroglu, N. J. Fraser, G. Gambardella, M. Blott, P. Leong, M. Jahre, and K. Vissers, “Finn: A framework for fast, scalable binarized neural network inference,” 2017, pp. 65–74.
- [2] X. Jia, Y. Zhang, G. Liu, X. Yang, T. Zhang, J. Zheng, D. Xu, H. Wang, R. Zheng, S. Pareek, L. Tian, D. Xie, H. Luo, and Y. Shan, “Xvdpu: A high performance cnn accelerator on the versal platform powered by the ai engine,” 2022, pp. 209–217.
- [3] N. Perryman, C. Wilson, and A. George, “Evaluation of xilinx versal architecture for next-gen edge computing in space,” vol. 2023-March, 2023.
- [4] T. Saidani, R. Ghodhban, A. Alhomoud, A. Alshammari, H. Zayani, and M. B. Ammar, “Hardware acceleration for object detection using yolov5 deep learning algorithm on xilinx zynq fpga platform,” *Engineering, Technology and Applied Science Research*, vol. 14, no. 1, pp. 13 066–13 071, 2024.
- [5] S. Lyu, Y. Zhao, X. Liu, Z. Li, C. Wang, and J. Shen, “Detection of male and female litchi flowers using yolo-hpfd multi-teacher feature distillation and fpga-embedded platform,” *Agronomy*, vol. 13, no. 4, 2023.
- [6] H.-B. Wang, Y.-S. Wang, J.-H. Xiao, S.-L. Wang, and T.-J. Liang, “Impact of single-event upsets on convolutional neural networks in xilinx zynq fpgas,” *IEEE Transactions on Nuclear Science*, vol. 68, no. 4, pp. 394–401, 2021.
- [7] I. Souvatzoglou, A. Papadimitriou, A. Sari, V. Vlagkoulis, and M. Psarakis, “Analyzing the single event upset vulnerability of binarized neural networks on sram fpgas,” in *IEEE International Symposium on Defect and Fault Tolerance in VLSI and Nanotechnology Systems (DFT)*, 2021, pp. 1–6.
- [8] B. Du, S. Azimi, C. de Sio, L. Bozzoli, and L. Sterpone, “On the reliability of convolutional neural network implementation on sram-based fpga,” in *IEEE International Symposium on Defect and Fault Tolerance in VLSI and Nanotechnology Systems (DFT)*, 2019, pp. 1–6.
- [9] F. Libano, B. Wilson, M. Wirthlin, P. Rech, and J. Brunhaver, “Understanding the impact of quantization, accuracy, and radiation on the reliability of convolutional neural networks on fpgas,” *IEEE Transactions on Nuclear Science*, vol. 67, no. 7, pp. 1478–1484, 2020.
- [10] W. Li, G. Ge, K. Guo, X. Chen, Q. Wei, Z. Gao, Y. Wang, and H. Yang, “Soft error mitigation for deep convolution neural network on fpga accelerators,” in *2nd IEEE International Conference on Artificial Intelligence Circuits and Systems (AICAS)*, 2020, pp. 1–5.
- [11] F. Libano, B. Wilson, J. Anderson, M. J. Wirthlin, C. Cazzaniga, C. Frost, and P. Rech, “Selective hardening for neural networks in fpgas,” *IEEE Transactions on Nuclear Science*, vol. 66, no. 1, pp. 216–222, 2019.
- [12] G. Jocher, *YOLOv5 by Ultralytics*, version 7.0, May 2020.
- [13] J. Ansel, E. Yang, H. He, N. Gimelshein, A. Jain, M. Voznesensky, B. Bao, P. Bell, D. Berard, E. Burovski, G. Chauhan, A. Chourdia, W. Constable, A. Desmaison, Z. DeVito, E. Ellison, W. Feng, J. Gong, M. Gschwind, B. Hirsh, S. Huang, K. Kalambarkar, L. Kirsch, M. Lazos, M. Lezcano, Y. Liang, J. Liang, Y. Lu, C. Luk, B. Maher, Y. Pan, C. Puhersch, M. Reso, M. Saroufim, M. Y. Siraichi, H. Suk, M. Suo, P. Tillet, E. Wang, X. Wang, W. Wen, S. Zhang, X. Zhao, K. Zhou, R. Zou, A. Mathews, G. Chanan, P. Wu, and S. Chintala, “PyTorch 2: Faster Machine Learning Through Dynamic Python Bytecode Transformation and Graph Compilation,” in *29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2 (ASPLOS)*, ACM, Apr. 2024.
- [14] J. Shermeyer, T. Hossler, A. Van Etten, D. Hogan, R. Lewis, and D. Kim, *Rareplanes dataset*, In-Q-Tel - CosmiQ Works and AIReverie, 2020.
- [15] AMD-Xilinx, *Versal Adaptive SoC Technical Reference Manual (AM011) v1.6*, <https://docs.amd.com/r/en-US/am011-versal-acap-trm>, Accessed: 2024-09-30, 2023.
- [16] —, *Versal: The First Adaptive Compute Acceleration Platform (ACAP)(WP505) v1.1.1*, <https://docs.amd.com/v/u/en-US/wp505-versal-acap>, Accessed: 2024-10-02, 2020.
- [17] —, *VCK190 Evaluation Board User Guide (UG1366) v1.2*, <https://docs.amd.com/r/en-US/ug1366-vck190-eval-bd>, Accessed: 2024-09-30, 2024.
- [18] —, *DPUCVDX8G for Versal ACAPs Product Guide (PG389) v1.3*, <https://docs.amd.com/r/en-US/pg389-dpucvdx8g>, Accessed: 2024-09-30, 2023.
- [19] —, *Vitis AI User Guide (UG1414) v3.0*, <https://docs.amd.com/r/3.0-English/ug1414-vitis-ai/Vitis-AI-Overview>, Accessed: 2024-09-30, 2023.
- [20] Martín Abadi, Ashish Agarwal, Paul Barham, Eugene Brevdo, Zhifeng Chen, Craig Citro, Greg S. Corrado, Andy Davis, Jeffrey Dean, Matthieu Devin, Sanjay Ghemawat, Ian Goodfellow, Andrew Harp, Geoffrey Irving, Michael Isard, Y. Jia, Rafal Jozefowicz, Lukasz Kaiser, Manjunath Kudlur, Josh Leven-

berg, Dandelion Mané, Rajat Monga, Sherry Moore, Derek Murray, Chris Olah, Mike Schuster, Jonathon Shlens, Benoit Steiner, Ilya Sutskever, Kunal Talwar, Paul Tucker, Vincent Vanhoucke, Vijay Vasudevan, Fernanda Viégas, Oriol Vinyals, Pete Warden, Martin Wattenberg, Martin Wicke, Yuan Yu, and Xiaoqiang Zheng, *TensorFlow: Large-scale machine learning on heterogeneous systems*, Software available from tensorflow.org, 2015.